

Aufgabe 2.2: Division und Rest

Da wir die Befehle `div`, `rem` und `mul` nicht nutzen dürfen, implementieren wir den Algorithmus der **wiederholten Subtraktion** (Repeated Subtraction). Das ist die einfachste Methode: $\frac{a}{b}$ ist die Anzahl, wie oft man b von a abziehen kann, bis der Rest kleiner als b ist.

a) Implementierung in Ripes (Call-by-Value)

Hier werden die Werte direkt in den Registern `a0` und `a1` übergeben.

Dateiname: `div_mod_val.s`

```
.text
.globl main

main:
    # Test: 20 / 3 = 6 Rest 2
    li a0, 20      # Dividend
    li a1, 3       # Divisor

    # Aufruf div(a, b)
    # Argumente sind schon in a0, a1
    # Wir sichern a0/a1, weil div sie verändert
    mv s0, a0
    mv s1, a1

    jal ra, my_div
    # Ergebnis (Quotient) ist jetzt in a0

    # Aufruf mod(a, b)
    mv a0, s0      # Stelle Dividend wieder her
    mv a1, s1      # Stelle Divisor wieder her
    jal ra, my_mod
    # Ergebnis (Rest) ist jetzt in a0

    # Ende
    li a7, 10
    ecall

# -----
# Unterprogramm: my_div
# Eingabe: a0 (Dividend), a1 (Divisor)
# Ausgabe: a0 (Quotient)
# Clobbers: t0
# -----
my_div:
    li t0, 0       # t0 = Counter (Quotient)

    # Sonderfall: Division durch 0 vermeiden (optional)
    beqz a1, div_end

div_loop:
    blt a0, a1, div_end # Wenn a0 < a1, sind wir fertig
    sub a0, a0, a1      # a0 = a0 - a1
    addi t0, t0, 1      # Counter++
    j div_loop

div_end:
    mv a0, t0         # Ergebnis in Return-Register a0
    ret

# -----
# Unterprogramm: my_mod
# Eingabe: a0 (Dividend), a1 (Divisor)
# Ausgabe: a0 (Rest)
# -----
my_mod:
    # Sonderfall: Modulo 0
    beqz a1, mod_end
```

```
mod_loop:
    blt a0, a1, mod_end # Wenn a0 < a1, ist a0 der Rest
    sub a0, a0, a1      # a0 = a0 - a1
    j mod_loop

mod_end:
    ret                # a0 enthält bereits den Rest
```

b) Call-by-Reference

Hier enthalten a0 und a1 nicht die Werte selbst, sondern die **Adressen** im Speicher, wo die Werte liegen. Wir müssen sie erst laden (lw).

Dateiname: div_mod_ref.s

```
.data
val_a: .word 20
val_b: .word 3

.text
.globl main

main:
    la a0, val_a    # Lade ADRESSE von a
    la a1, val_b    # Lade ADRESSE von b

    jal ra, div_ref # Aufruf

    # Ergebnis in a0
    li a7, 10
    ecall

# -----
# Unterprogramm: div_ref
# Eingabe: a0 (Adresse von a), a1 (Adresse von b)
# Ausgabe: a0 (Quotient als Wert)
# -----
div_ref:
    # 1. Werte aus dem Speicher holen (Dereferenzierung)
    lw t0, 0(a0)    # t0 = Wert von a
    lw t1, 0(a1)    # t1 = Wert von b

    li a0, 0        # a0 nutzen wir als Counter/Ergebnis

    beqz t1, ref_end # Check div by zero

ref_loop:
    blt t0, t1, ref_end
    sub t0, t0, t1
    addi a0, a0, 1
    j ref_loop

ref_end:
    ret
```